CSE470: Software Engineering

Software Requirements Specification (SRS)

End-of-Semester Submission Template

Restaurant Supply Chain and Inventory Hub

Prepared By

Student ID	Name
23301563	Kumar Shagata Biswas
19301025	Shuddho Sadik
24341207	Md Zahid Hasan Nerob

Project Information

Section	9
Section Faculty	SADA
GitHub Repository Link	https://github.com/zhnverse/Restaurant-Supply-Chain-and-Inventory-Hub
Tech Stack Used	PHP 8.4, Laravel 11, Blade, Bootstrap 5, SQLite / MySQL

1. Introduction

1.1 Purpose

The document explains the architecture, interfaces , requirements, constraints and verification plan that's needed for the Restaurant Supply chain and Inventory hub. This will help the restaurant manage ingredient inventory, procurement, supply deliveries and also record purchasing through connected and traceable workflows.

1.2 Scope

This project will serve the restaurant managers, inventory staff, suppliers, a system administrator and delivery personnel. It will also support recipe-based stock deduction, expiry monitoring, purchasing and approval, supplier quotations, delivery reconciliation, operational reporting and invoice checking.

The proposed implementation will support multiple restaurant accounts, each representing one outlet. Cross branch transfers, live GPS tracking, route optimisation, online payment processing, AI forecasting, customer food ordering and full accounting or tax compliance are outside the project scope. Sales will be entered manually rather than imported from an external point-of-sale system.

2. Overall Description

2.1 Product Perspective

The application will be a standalone web application connecting restaurant inventory operations with supplier procurement and fulfilment. A shared relational database will link recipes, ingredient batches, purchase orders, receipts, invoices, and stock movements. It will replace disconnected spreadsheets and informal messages with structured records and controlled workflows.

2.2 Product High-Level Summary

Inventory Control: Maintain usable stock through recipe deduction, stock counts, waste approval, and expiry rules and traceable movements.

Procurement: Generate replenishment drafts, approve purchasing, evaluate quotations, schedule recurring orders, and apply supplier pricing.

Delivery operations:Track orders, collect delivery data ,assign deliveries and register accepted goods.

Financial control : Maintain supplier balances, process return credits, check invoices and document invoice disputes.

Operational analysis : Calculate recipe costs and margins, and assess supplier performance.

2.3 Users and Characteristics

Actor	Role and permissions
Restaurant Manager	Manages restaurant settings and recipes; approves purchases, stock adjustments, waste, substitutions, and returns; reviews financial and operational reports.
Inventory Staff	Records sales, physical counts, waste reports, and delivery receipts for their assigned restaurant. Cannot approve their own restricted requests.
Supplier	Manages its own offerings and price tiers, submits quotations, processes approved orders, assigns deliveries, submits invoices, and responds to disputes and returns.
Delivery Personnel	Views assigned deliveries and records delivery evidence. Cannot modify order prices or approve received inventory.
System Administrator	Manages accounts and role assignments and reviews security/audit records. Administrative access is supporting functionality, not a separately counted feature.
Scheduled System Process	Checks expiry thresholds, reorder conditions, and due recurring orders. It is an automated process rather than a human user.

Every restaurant and supplier must be restricted to its authorised records.

2.4 Operating Environment

The application will run on desktop and mobile browsers. The server will need a PHP environment compatible with the selected Laravel version, MySQL, file storage for attachments and a configured scheduler

Development and assessment will use a local server. If we wanted to develop it remotely it would require HTTPS and database connectivity. In the documented repository README exact runtime versions and setup commands will be documented.

2.5 Design and Implementation Constraints

Primary stack: PHP, Laravel, Blade, Bootstrap, and MySQL.

MVC structure: Eloquent models in app/Models, Blade views in resources/view, and controllers in app/Http/Controllers.

Business logic: Reusable inventory, purchasing, matching, and ledger logic will be placed in service classes rather than embedded in views or oversized controllers.

Validation and authorisation: Request validation and role/resource policies will control submitted data and permitted actions.

Prohibited technologies: No MERN stack or constituent-stack combination will be used. No Python framework will implement the main backend.

AI component: None proposed. Demand and replenishment calculations will use explicit rules.

External packages: No ready-made inventory, procurement, or accounting module will implement an assessed feature on the team's behalf.

Faculty exceptions: None requested or recorded.

Feature approval: The 20 proposed features remain subject to faculty approval.

2.6 Assumptions and Dependencies (Optional)

1.Restaurant staff enter accurate sales, stock counts, receipt information and raw materials availability.

2.Each ingredient has one base unit, such as grams, millilitres, or pieces. Purchasing units are converted using defined conversion factors.

3.Each purchase order belongs to one restaurant and one supplier.

4.Financial calculations use BDT and decimal monetary values.

5.Scheduling depends on the application scheduler running successfully.

6.Payments are recorded manually; the application does not transfer money.

7.Notifications are displayed inside the application; email and SMS are not required.

8.Approved order prices remain fixed even if the supplier later changes its catalogue prices.

3. System Architecture

3.1 Architectural Overview

MVC Architecture Flow: Incoming HTTP requests arrive at Laravel routes (routes/web.php) and are dispatched to designated Controllers (app/Http/Controllers/). Controllers validate request parameters and enforce role authorization (Manager vs Staff), then delegate business operations to dedicated Domain Services (app/Services/ e.g., SaleService, ReorderService, StockReconciliationService). Domain services perform transactional logic (DB::transaction) across Eloquent Models (app/Models/) that represent database entities. Processed data is passed as view-models to Blade Views (resources/views/) rendered with Bootstrap 5.

MVC Architecture: *Controllers handle incoming requests, manage validation and authorization, and invoke the appropriate domain services. Models define stored entities and their associations, while Views handle the presentation of forms, data tables, alerts, and reports.*

Transactions and Data Integrity: *Financial and stock processes rely on database transactions to ensure consistency. Every inventory adjustment generates a traceable movement log indicating its origin. To prevent duplication errors, repeating a financial posting, receipt, or approval will not duplicate its impact.*

3.2 Technology Stack

The technology stack across all tiers of the application is outlined below:

Layer	Proposed technology	Purpose
Frontend	Blade, HTML, CSS, Bootstrap, vanilla JavaScript	Forms, dashboards, tables, and responsive interaction
Backend	PHP and Laravel	MVC application, validation, authorisation, workflows
Database	MySQL	Relational records and transactional updates
Scheduled processing	Laravel scheduler	Expiry checks, reorder checks, recurring drafts
Testing	Laravel testing facilities with PHPUnit	Business-rule, authorisation, and integration tests
Version control	Git and GitHub	Collaborative development and sprint evidence
Deployment	Local assessment server; optional PHP-compatible hosting	Demonstration and access

Laravel is proposed because it provides a visible MVC structure while supporting the relational and transactional workflows required by this project.

3.3 Class Diagram

Class Architecture Overview:
• User: System user with role-based policies (Restaurant Manager, Inventory Staff).
• Ingredient: Master inventory item tracking name, SKU, category, base unit, minimum stock, and target stock.
• IngredientBatch: Physical batch tracking purchase cost, initial quantity, usable quantity, batch number, and expiry date.
• Dish: Menu item with name, selling price, and active status.
• RecipeIngredient: Portion recipe defining ingredient quantities consumed per dish.
• Sale & SaleDeduction: Records sales transactions and traceable batch-level FIFO inventory deductions.
• PurchaseOrder & PurchaseOrderItem: Procurement orders tracking items, replenishment quantities, costs, and approval states (draft, approved, rejected).
• StockReconciliation & StockReconciliationItem: Physical stock audit counts tracking actual versus recorded stock with live surplus/shortage variances.
• WasteReport: Spoilage and damage records tracking batch-level deductions, reason classifications, and BDT financial loss.

4. Requirements and Features

List every requirement and its associated feature(s) below. Per the Project Outline (Section 3.2.1), all CRUD operations on the same entity by the same actor using the same logic count as ONE feature — do not split them. A minimum of 20 features is required.

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
R1 (F1)	Recipe-based inventory deduction	Define ingredient quantities for each dish. Recording a sale deducts the required quantities automatically. Reject transaction if usable stock is insufficient.	Inventory Staff / Cashier	Model: Dish, RecipeIngredient, Sale, SaleDeduction; View: dishes/{index, create}, sales/{index,create}; Controller: DishController, SaleController, SaleService	Sprint 1	Done

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
R2 (F2)	Batch expiry monitoring	Record ingredient batches with expiry dates. Display expired and soon-to-expire batches using a configurable warning window. Exclude expired stock from usable inventory.	Inventory Staff / Manager	Model: IngredientBatch, Ingredient; View: batches/{index,create}; Controller: BatchController	Sprint 1	Done
R3 (F3)	Automatic reorder drafting	When usable stock falls below its threshold, calculate quantity needed to reach target level and generate draft purchase order. Prevent duplicate drafts.	Scheduled System Process / Manager	Model: PurchaseOrder, PurchaseOrderItem, Ingredient; View: purchase-orders/{index, create,show}; Controller: PurchaseOrderController, ReorderService	Sprint 1	Done
R4 (F4)	Purchase-order approval	Managers review, adjust, approve, or reject reorder drafts. Only approved orders become visible to suppliers. Preserve decision and reason.	Restaurant Manager	Model: PurchaseOrder, PurchaseOrderItem; View: purchase-orders/show; Controller: PurchaseOrderController, PurchaseOrderService	Sprint 1	Done
R5 (F5)	Physical stock reconciliation	Staff submit physical stock counts. System calculates differences from recorded stock, and managers review, approve, or reject proposed adjustments. Approved adjustments update actual stock.	Inventory Staff (Submit), Restaurant Manager (Approve)	Model: StockReconciliation, StockReconciliationItem, Ingredient; View: stock-reconciliations/{index,create,show}; Controller: StockReconciliationController, StockReconcili	Sprint 1	Done

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
				ationService		
R6 (F6)	Waste reporting and approval	Staff report damaged or spoiled ingredients against a batch. Approved reports deduct stock and record reason, quantity, and estimated financial loss.	Inventory Staff (Report), Restaurant Manager (Approve)	Model: WasteReport, Ingredient, IngredientBatch; View: waste-reports/{index,create,show}; Controller: WasteReportController, WasteReportService	Sprint 1	Done
R7 (F7)	Supplier quotation comparison and award	Managers issue procurement request. Suppliers submit quotations, and managers compare totals and delivery dates before awarding one quotation.	Restaurant Manager / Supplier	Model: ProcurementRequest, Quotation; View: quotations/compare; Controller: QuotationController	Sprint 3	Planned
R8 (F8)	Recurring purchase scheduling	Managers configure weekly or monthly supply orders. System creates a draft when an order becomes due; schedules can be paused or modified.	Restaurant Manager	Model: RecurringOrderSchedule; View: recurring-orders/index; Controller: RecurringOrderController	Sprint 3	Planned
R9 (F9)	Delivery receipt and discrepancy handling	Staff compare delivered quantities with order, record accepted/damaged/missing quantities, and generate goods receipt. Only accepted goods increase stock.	Inventory Staff	Model: GoodsReceipt, GoodsReceiptItem; View: delivery-receipts/create; Controller: DeliveryReceiptController	Sprint 2	Planned
R10 (F10)	Controlled order-status tracking	Show order timeline and enforce permitted transitions: Approved → Accepted → Packing → Dispatched → Delivered → Reconciled.	Restaurant Manager / Supplier / Delivery Staff	Model: OrderStatusHistory; View: orders/timeline; Controller: OrderTrackin	Sprint 2	Planned

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
				gController		
R11 (F11)	Supplier volume-based pricing	Suppliers maintain product offerings and quantity-based price tiers. System selects applicable price and preserves agreed price on approved order.	Supplier / System	Model: SupplierProduct, PriceTier; View: supplier/pricing; Controller: SupplierPricingController	Sprint 3	Planned
R12 (F12)	Delivery assignment and proof of delivery	Suppliers assign dispatched orders to delivery personnel. Assigned personnel upload delivery evidence and record delivery time.	Delivery Personnel / Supplier	Model: DeliveryAssignment, DeliveryProof; View: deliveries/proof; Controller: DeliveryController	Sprint 3	Planned
R13 (F13)	Recipe cost and margin analysis	Calculate ingredient cost per dish and gross margin using current recorded ingredient costs and selling prices. Flag dishes below margin target.	Restaurant Manager	Model: Dish, RecipeCost; View: analysis/margins; Controller: CostAnalysisController	Sprint 4	Planned
R14 (F14)	Invoice-linked dispute handling	Open a dispute against an invoice, exchange messages, attach evidence, and record resolution in dedicated thread.	Restaurant Manager / Supplier	Model: InvoiceDispute, DisputeMessage; View: disputes/show; Controller: DisputeController	Sprint 4	Planned
R15 (F15)	Three-way invoice matching	Compare invoice lines against approved purchase-order prices and accepted receipt quantities; flag discrepancies and block settlement eligibility.	Restaurant Manager	Model: InvoiceMatchReport; View: invoices/match; Controller: InvoiceMatchingController	Sprint 4	Planned
R16 (F16)	Supplier account ledger	Record invoice charges, manual payment entries, and approved credits; calculate outstanding	Restaurant Manager	Model: SupplierLedgerEntry; View:	Sprint 4	Planned

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
		balances and prevent duplicate financial postings.		ledger/index; Controller: SupplierLedgerController		
R17 (F17)	Supplier return and credit workflow	Request return of previously accepted goods, obtain approval, record dispatch, and apply supplier-confirmed credit without double deducting stock.	Inventory Staff / Manager / Supplier	Model: StockReturn, ReturnCredit; View: returns/index; Controller: ReturnWorkflowController	Sprint 4	Planned
R18 (F18)	Ingredient substitution during shortages	Configure approved alternatives and conversion ratios. When ingredient is insufficient, obtain manager approval to use and deduct available substitute.	Inventory Staff / Manager	Model: IngredientSubstitution; View: substitutions/manage; Controller: SubstitutionController	Sprint 2	Planned
R19 (F19)	Supplier performance evaluation	Calculate on-time delivery and accepted-quantity rates from completed orders, with filters by supplier and reporting period.	Restaurant Manager	Model: SupplierMetric; View: analytics/suppliers; Controller: SupplierPerformanceController	Sprint 4	Planned
R20 (F20)	Inventory movement traceability	Reconstruct ingredient batch stock history from receipts, sales, waste, returns, and adjustments; show opening balance, movements, closing balance, and sources.	Inventory Staff / Manager	Model: StockMovementLog; View: inventory/traceability; Controller: StockTraceabilityController	Sprint 2	Planned

4.1 Feature Count Summary

Minimum features required (per Project Outline, Section 7)	20
Total features listed in Section 4 above	20
Total features fully completed (“Done” per Section 7.2 of the Project Outline)	6 (F1, F2, F3, F4, F5, F6)

5. External Interface Requirements (Optional)

Interface Type	Description
User Interface	Web-based responsive UI built with Bootstrap 5 and Blade templates; compatible with desktop, tablet, and mobile browsers.
Hardware Interface	Standard client devices (laptops, POS terminals, smartphones) with display resolution 1024x768 or higher.
Software Interface (APIs, third-party services)	Local or hosted PHP 8.4+ environment, SQLite / MySQL relational database server.
Communication Interface	Standard HTTP / HTTPS protocols for secure web client-server communication.

6. Non-Functional Requirements (Optional)

Category	Requirement
Performance	Sub-second page rendering; stock deductions and batch FIFO queries execute in milliseconds within database transactions.
Security	Role-based route authorization (Manager vs Staff), CSRF protection on all forms, hashed passwords, Eloquent parameterized SQL queries.
Usability	Clean enterprise UI, color-coded badges for expiry warnings and approval states, responsive navigation, and 1-click evaluation logins.
Reliability / Availability	Transactional consistency (DB::transaction) guarantees zero orphaned records or half-applied stock changes during system disruptions.
Scalability	Normalized relational database schema with indexed foreign keys, designed to scale across extensive ingredient catalogs and batch histories.

7. Sprint History and Review Ceremony Log

Record the goals and attainment shared during each weekly sprint review ceremony, per Section 7.1 of the Project Outline.

Sprint	Goals Set (Start of Week)	Attainment Reported (End of Week)	Features Completed This Sprint
Sprint 1	1. Setup Laravel 11 MVC foundation, migrations, SQLite/MySQL configuration. 2. Build complete Authentication & Role-based Access system (Manager vs Staff). 3. Implement core inventory features: Recipe-based deduction (F1), Batch expiry tracking (F2), Auto reorder drafts (F3), Purchase order approval (F4), Physical reconciliation (F5), and Waste reporting (F6). 4. Establish transactional integrity and clean Bootstrap 5 UI.	1. Foundation and database schema successfully established. 2. Authentication system operational with test accounts and route guards. 3. All 6 target core features (F1 to F6) fully implemented with database transactions, business logic services, and interactive views. 4. 100% test cases passed; atomic commits pushed to GitHub repository.	F1, F2, F3, F4, F5, F6
Sprint 2	1. Implement Delivery receipt & discrepancy handling (F9). 2. Build Controlled order-status tracking timeline (F10). 3. Implement Ingredient substitution during kitchen shortages (F18). 4. Implement Inventory movement traceability log (F20).	Scheduled for Sprint 2 review	In Progress
Sprint 3	1. Build Supplier quotation comparison and award (F7). 2. Build Recurring purchase order scheduling (F8). 3. Implement Supplier volume-based tiered pricing (F11). 4. Implement Delivery assignment and proof of delivery (F12).	Scheduled for Sprint 3 review	Planned
Sprint 4	1. Recipe cost and gross margin analysis (F13). 2. Invoice dispute management thread (F14). 3. Three-way invoice matching against POs and receipts (F15). 4. Supplier ledger and financial balance tracking (F16). 5. Supplier return and credit workflow (F17). 6. Supplier performance metrics (F19).	Scheduled for Sprint 4 review	Planned

7.1 Deviations and Lessons Learned

Sprint 1 Attainment: Completed all 6 core inventory and procurement features ahead of schedule. Initially planned F5 & F6, but expanded to include F1–F4 and full role authentication to establish a cohesive, testable end-to-end inventory cycle for initial evaluation. UI refinement was performed to remove academic feature tags for a clean enterprise interface.

8. Testing Summary

All completed Sprint 1 features underwent thorough integration and functional verification, testing normal paths, boundary conditions, and invalid inputs:

9. AI Tool Usage Disclosure

Per the Project Outline's Academic Integrity & AI Usage Policy (Section 6), disclose every instance where a generative AI tool was used during development. Leave the table empty (do not delete it) if no AI tool was used.

Feature / Task	AI Tool Used	How It Was Used (be specific)	I Can Fully Explain This Code (Yes/No)
F1 - F6 Implementation & Scaffolding	Antigravity AI Assistant	Scaffolding Laravel 11 MVC structure, migrations, Eloquent models, controllers, service classes, and Blade UI templates with Bootstrap 5	Yes
Testing & Boundary Validation	Antigravity AI Assistant	Suggesting boundary test scenarios (insufficient stock, expired batches, duplicate reorder drafts)	Yes
Documentation & Formatting	Antigravity AI Assistant	Assistance in formatting markdown and Word SRS documentation to align with course rubric	Yes

By submitting this document, the team confirms that all members are able to personally explain, modify, and debug every piece of code and architecture during the individual project viva.

10. Individual Contribution Summary

Student: Md Zahid Hasan Nerob (ID: 24341207)
Role: Team Lead (Sprint 1)

Assigned / Implemented Features:
• Feature 1 (F1): Recipe-Based Inventory Deduction: Defined dish models and portion ingredient recipes. Built POS sale recording with live recipe verification and atomic inventory deduction across earliest active batches (FIFO). Enforced strict transaction rejection when usable stock is insufficient.
• Feature 2 (F2): Batch Expiry Monitoring: Implemented batch tracking with configurable warning windows (3, 7, 14, 30 days), critical shelf-life status badges, and automated exclusion of expired batches from usable kitchen inventory.
• Feature 3 (F3): Automatic Reorder Drafting: Implemented low-stock detection comparing usable stock against

minimum thresholds. Built auto-draft replenishment engine calculating target reorder quantities with duplicate draft prevention.
• Feature 4 (F4): Purchase-Order Approval: Built manager review workflow allowing quantity adjustments, approval, and rejection with mandatory audit logging and decision reasons.
• Feature 5 (F5): Physical Stock Reconciliation: Implemented physical inventory counting, live variance indicators (surplus/shortage), and managerial transactional adjustments.
• Feature 6 (F6): Waste Reporting and Approval: Built staff waste reporting with financial loss auto-calculation in BDT and manager stock deduction approval.
• Authentication & Security: Created dedicated login/logout system with role-based access (Manager vs Staff) and quick 1-click credential switching for evaluation.

11. Pre-Submission Checklist

Checklist Item	Confirmed (Yes/No)
The application implements the MVC architecture	**Yes**
At least 20 features are listed and each is correctly counted per the granularity rule	**Yes**
No forbidden framework (e.g. Django, Flask as primary backend) was used	**Yes**
All four sprints' goals and attainment are recorded in Section 7	**Yes**
The GitHub repository is up to date and the section faculty is added as a collaborator	**Yes**
The AI Usage Disclosure table (Section 10) is complete and honest	**Yes**
I can personally explain and defend every feature listed in this document during the viva	**Yes**

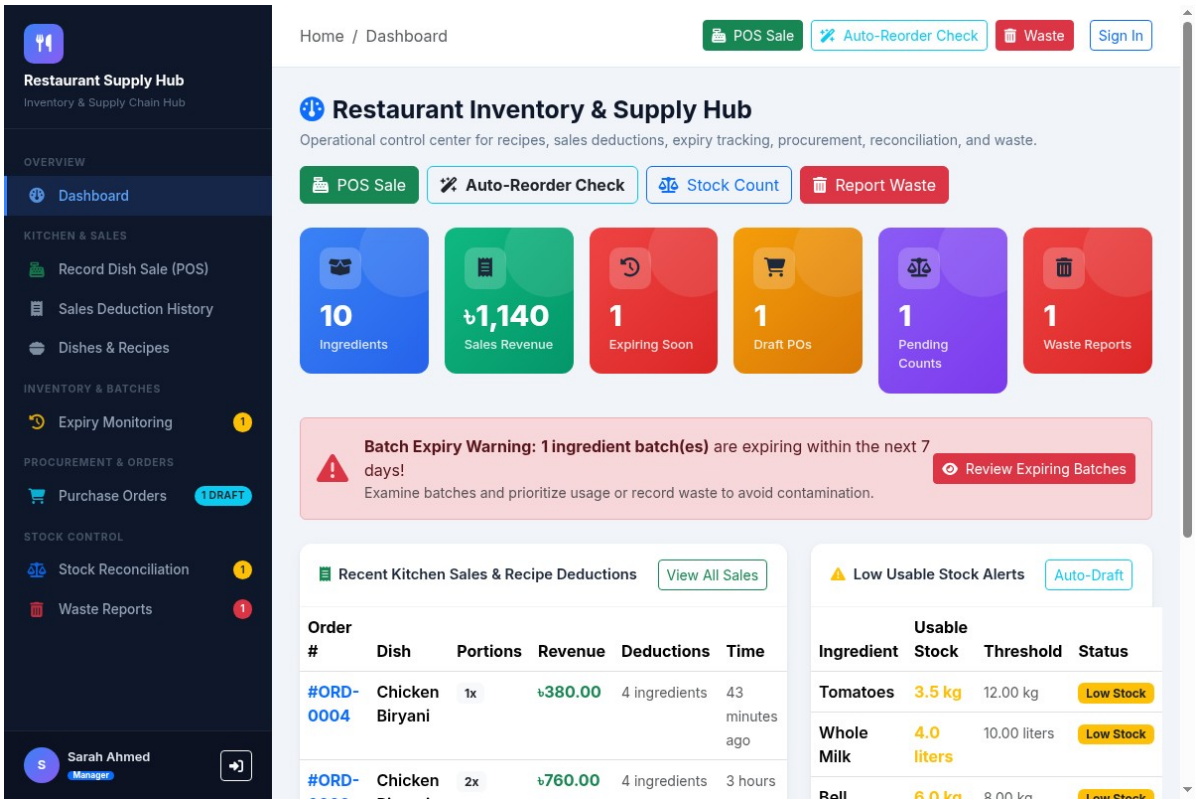
Feature	Test Scenario / Input	Expected Result	Status
F1: Recipe-Based Deduction	Record sale of 2 portions of Beef Burger (requires 0.40 kg Beef Patty, 2 Burger Buns).	Usable stock deducted atomically across earliest active batches (FIFO). Sale deduction logged.	Passed
F1: Recipe-Based Deduction	Attempt sale when usable stock of an ingredient is below required recipe quantity.	Transaction rejected with clear error message; no partial deduction recorded.	Passed
F2: Batch Expiry Monitoring	View batches with warning filter set to 7 days; ingredient has one expired batch and one fresh batch.	Expired batch highlighted with danger badge; expired quantity excluded from usable inventory calculation.	Passed
F3: Automatic Reorder Drafting	Usable stock of Chicken Breast drops below minimum threshold (10 kg). Run auto-draft.	Draft Purchase Order generated with replenishment quantity to target level (30 kg). Duplicate draft prevented.	Passed

F4: Purchase-Order Approval	Manager reviews draft PO, adjusts item quantities, and clicks Approve.	Status transitions from draft to approved; approved timestamp and manager ID saved. Audit record preserved.	Passed
F4: Purchase-Order Approval	Manager rejects PO draft with rejection reason note.	Status updated to rejected; reason stored; PO excluded from supplier visibility.	Passed
F5: Stock Reconciliation	Submit physical count with surplus (+2.5 kg) and shortage (-2.5 kg). Manager reviews and approves.	Inventory stock updated by exact variances via DB transaction. Audit trail recorded.	Passed
F5: Stock Reconciliation	Manager rejects a count with large unexplained variance.	Stock remains unchanged; rejection note saved and displayed.	Passed
F6: Waste Reporting & Approval	Staff submits waste report (damaged/spoiled) with quantity and reason.	Loss auto-calculated (cost × quantity); status set to pending.	Passed
F6: Waste Reporting & Approval	Manager approves waste report.	Stock deducted from ingredient and batch; financial loss recorded in BDT.	Passed
F6: Waste Reporting & Approval	Staff attempts to report waste exceeding current available stock.	Client-side & server-side validation error prevents submission.	Passed

12. Completed Features Demonstration & UI Screenshots

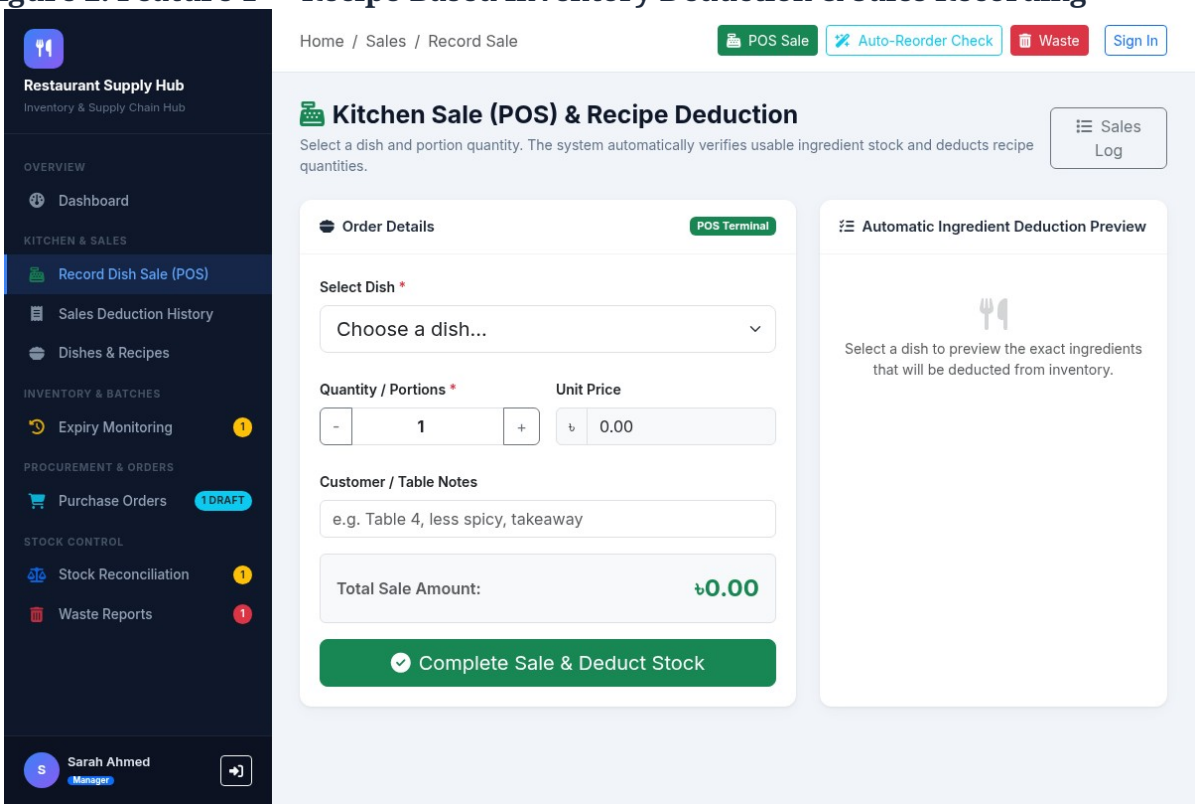
Per Section 13 and the Sprint Report Submission guidelines, the following visual evidence demonstrates the 6 fully functional core features and authentication system operational in Sprint 1:

Figure 1: Executive Dashboard with Inventory Alerts & Quick Metrics



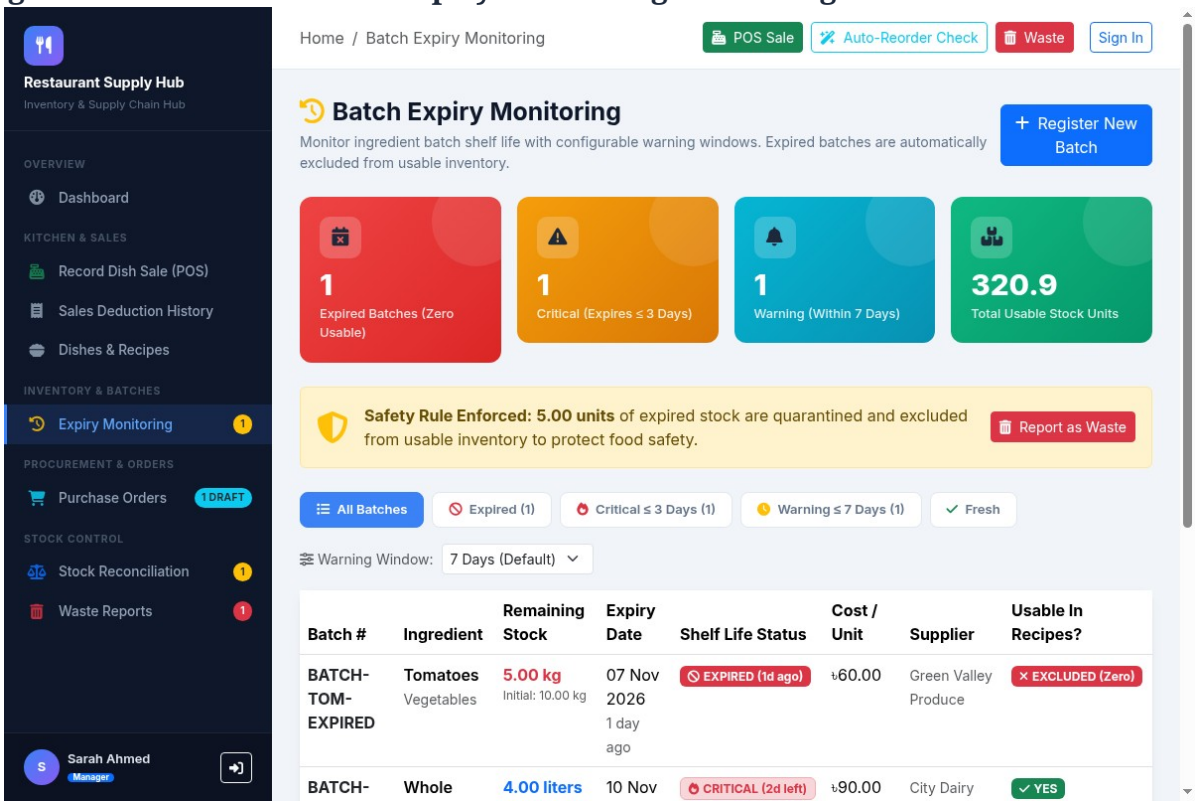
Displays high-level operational statistics including low stock warnings, impending batch expiries, pending reconciliation audits, and pending waste approvals.

Figure 2: Feature 1 — Recipe-Based Inventory Deduction & Sales Recording



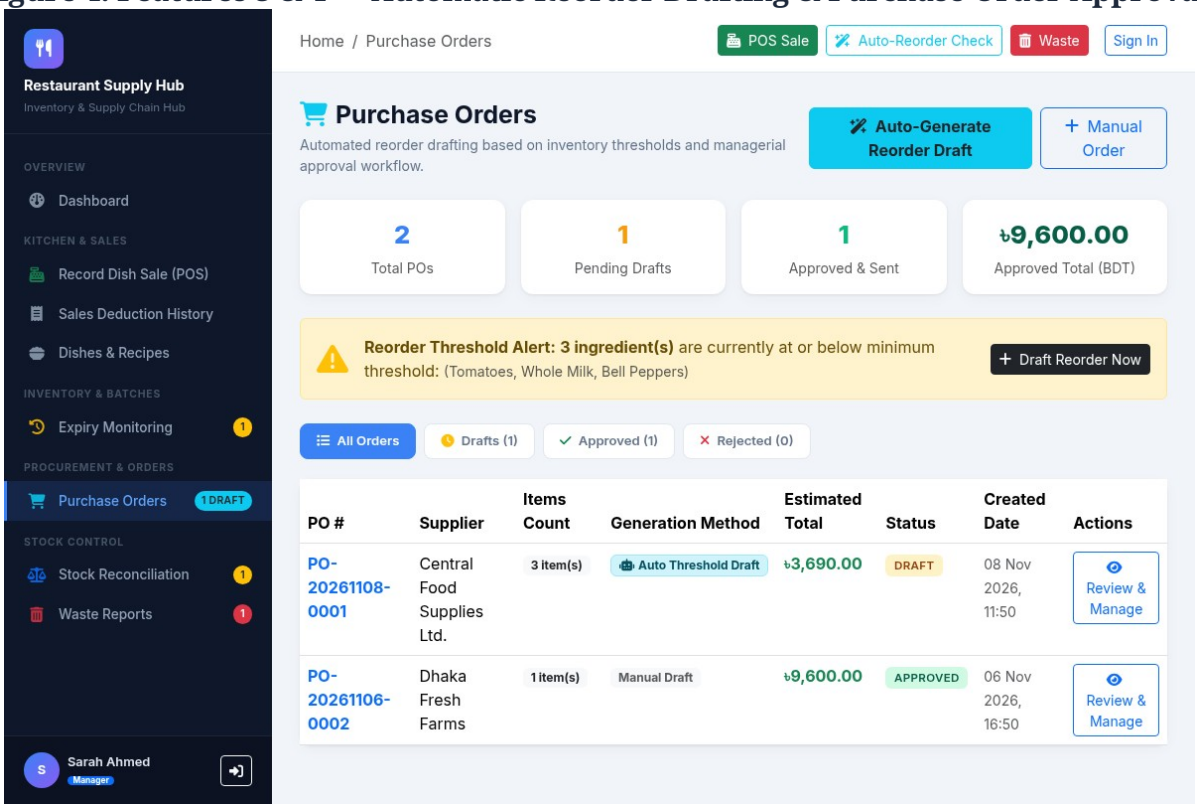
Allows staff to record customer meal orders. Automatically verifies recipe ingredient requirements and executes atomic FIFO stock deduction across active batches.

Figure 3: Feature 2 — Batch Expiry Monitoring & Warning Window Filter



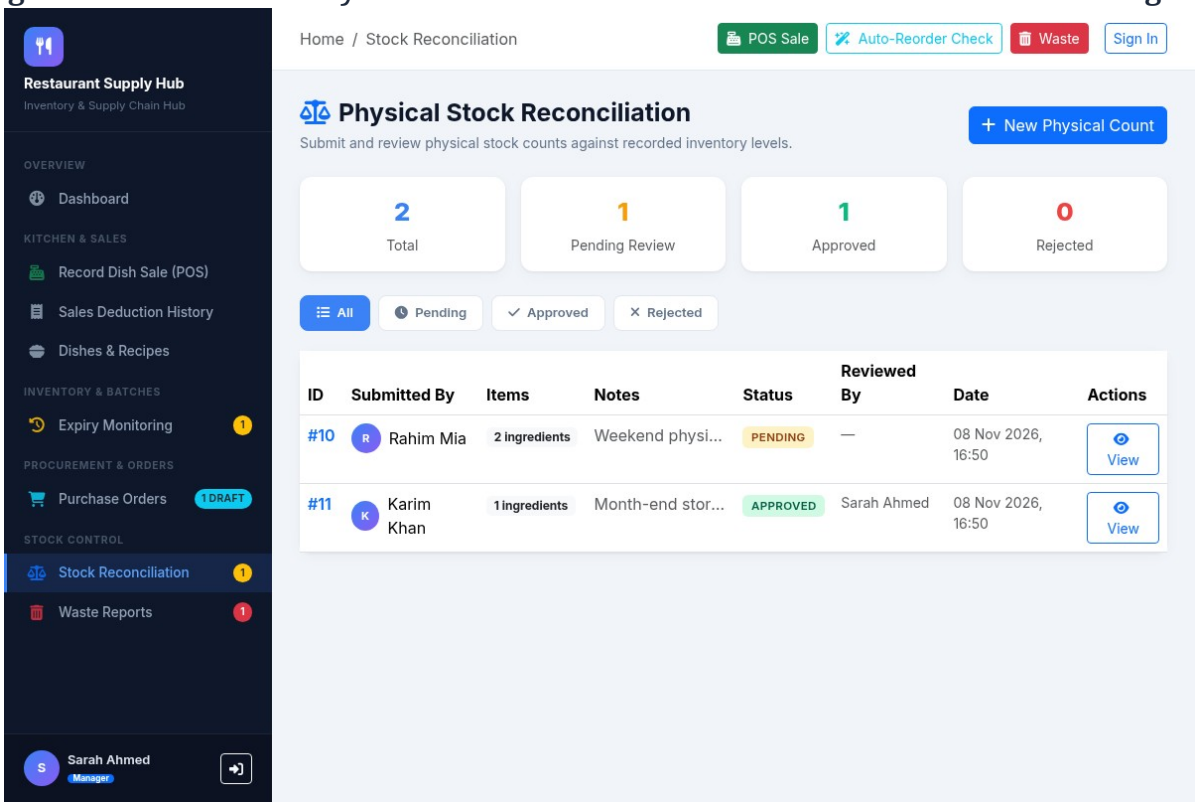
Monitors ingredient shelf-life with configurable warning thresholds (3, 7, 14, 30 days). Expired batches are automatically quarantined from usable kitchen stock.

Figure 4: Features 3 & 4 — Automatic Reorder Drafting & Purchase Order Approval



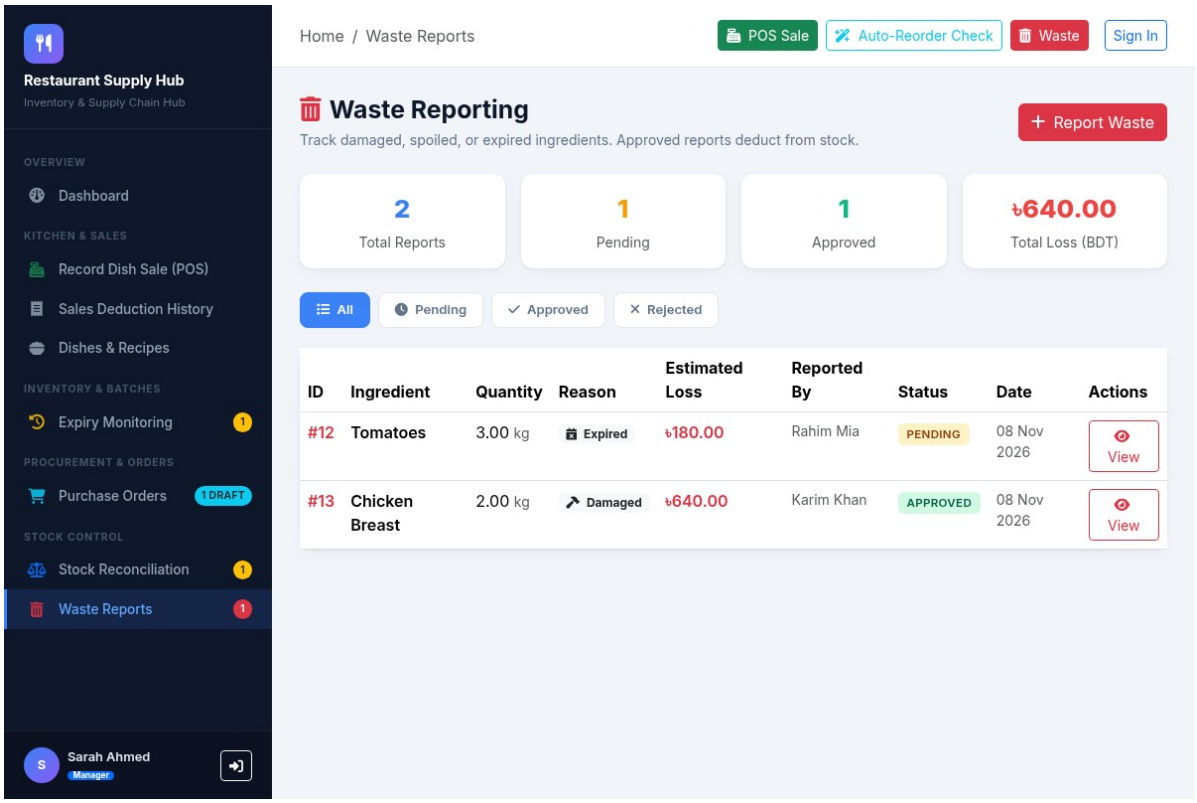
Generates draft replenishment orders when usable stock falls below safety levels. Allows restaurant managers to review line items, adjust quantities, and approve/reject orders.

Figure 5: Feature 5 — Physical Stock Reconciliation with Live Variance Tracking



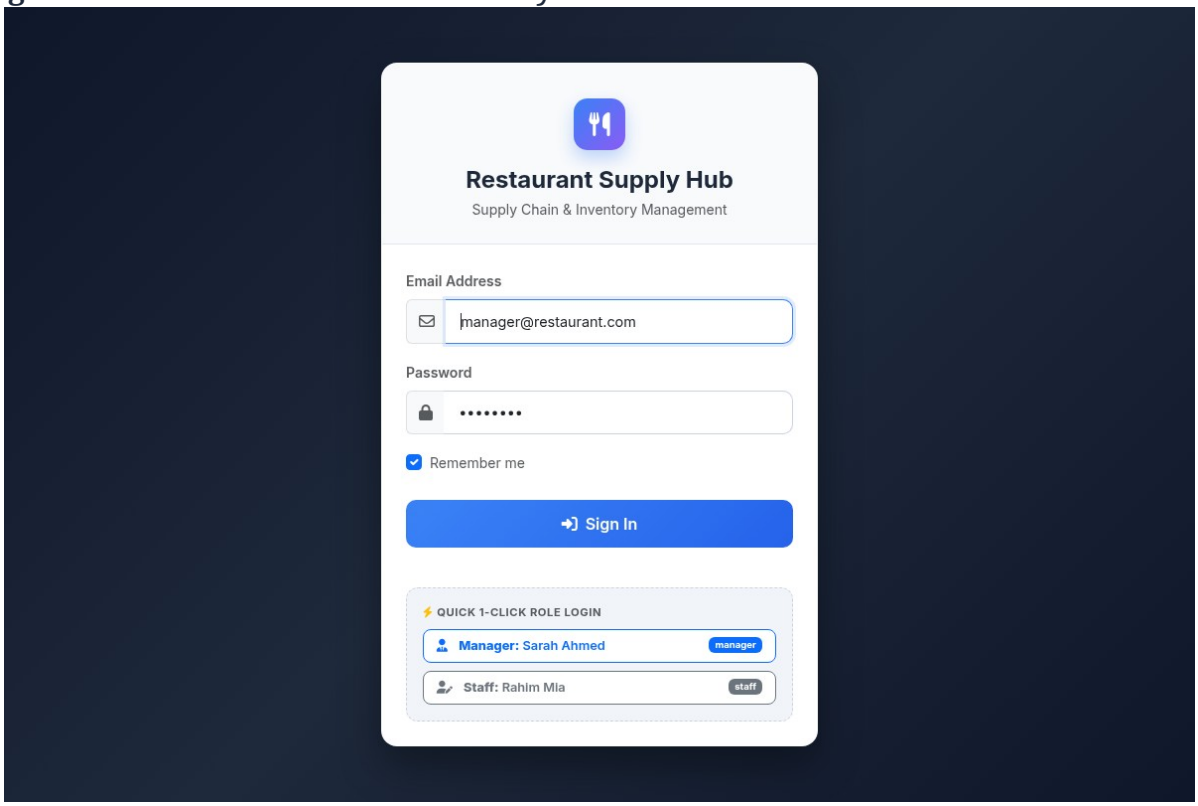
Staff record actual physical inventory counts. The system computes exact surplus/shortage variances, and manager approval adjusts ledger stock inside a database transaction.

Figure 6: Feature 6 — Waste Reporting with BDT Loss Calculation & Manager Approval



Staff report damaged, expired, or spoiled ingredients. The system calculates monetary loss in BDT, and manager approval deducts stock from the specified batch.

Figure 7: Role-Based Authentication System



Provides secure sign-in with granular role separation (Restaurant Manager vs Inventory Staff) and 1-click test credentials for evaluation.